

Design and Implementation of an ETL Pipeline Using Talend and SQL Server for Multi-Source Data Integration

ID/X Partners – Data Engineer

Presented by
Ryan Besto Saragih

About Company

The logo for ID/X Partners, consisting of the text "id/x" in blue on a white background, followed by "partners" in white on a dark blue background.**id/x partners**

ID/X Partners is a technology consulting firm focused on strategic partnerships to help clients achieve digital transformation. They provide innovative solutions across various domains, including data and analytics. This Data Warehouse project is an example of how ID/X Partners supports its clients in the banking industry to modernize their data infrastructure and address real business challenges.

Project Overview: Data Centralization for Accelerated Analysis

Background

This project is a case simulation for one of ID/X Partners' strategic clients operating in the banking sector. The client is facing a common challenge where their operational data including customer, account, and transaction information—is not integrated and is stored across multiple, separate systems.

Problem Statement (Key Issue)

- **Problem :** The data extraction process is manual and inefficient, involving various sources such as Excel files, CSVs, and databases, each operating independently.
- **Business Impact :** There are significant delays in report generation and data analysis processes. This hinders the company's ability to make timely, data-driven business decisions.

Project Overview: Data Centralization for Accelerated Analysis

Available Data Sources

To address this issue, eight main data sources are provided, divided into two types:

File-Based : transaction_excel.xlsx dan transaction_csv.csv

Database (SQL Server) : transaction_db, account, customer, branch, city, dan state

Project Objectives

As a Data Engineer, the main objectives of this project are:

- To build a centralized Data Warehouse (DWH) as a single source of truth for analytical data.
- To design and implement an automated ETL pipeline that extracts, cleans, and integrates data from all sources into the DWH.
- To provide fast access to data summaries through Stored Procedures to accelerate daily reporting and analysis.

Laying the Foundation – DWH Schema Design

Objective

The first task was to design and build a "new home" for the data to be analyzed. I created a new database named DWH in SQL Server and defined its table schema based on the Star Schema methodology.

Schema Design

The schema consists of three Dimension Tables (describing who and where) and one Fact Table (recording what happened).

- Dimension Tables: DimAccount, DimBranch, DimCustomer
- Fact Table: FactTransaction

Relationships between tables are clearly defined using Primary Keys and Foreign Keys to maintain data integrity.

DWH Schema Implementation with SQL

```
CREATE DATABASE DWH;  
GO
```

```
USE DWH;  
GO
```

```
CREATE TABLE DimAccount (  
    AccountID INT PRIMARY KEY,  
    CustomerID INT,  
    AccountType VARCHAR(50),  
    Balance MONEY,  
    DateOpened DATE,  
    Status VARCHAR(50)  
);
```

```
CREATE TABLE DimBranch (  
    BranchID INT PRIMARY KEY,  
    BranchName VARCHAR(100),  
    BranchLocation VARCHAR(255)  
);
```

```
CREATE TABLE DimCustomer (  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(100),  
    Address VARCHAR(255),  
    CityName VARCHAR(100),  
    StateName VARCHAR(100),  
    Age INT,  
    Gender VARCHAR(10),  
    Email VARCHAR(100)  
);
```

```
CREATE TABLE FactTransaction (  
    TransactionID INT PRIMARY KEY,  
    AccountID INT,  
    TransactionDate DATETIME,  
    Amount MONEY,  
    TransactionType VARCHAR(50),  
    BranchID INT,
```

```
FOREIGN KEY (AccountID) REFERENCES DimAccount(AccountID),  
FOREIGN KEY (BranchID) REFERENCES DimBranch(BranchID)
```

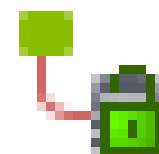
```
);
```

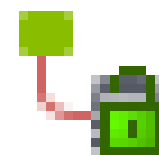
-   DWH
 -   Database Diagrams
 -   Tables
 -   System Tables
 -   FileTables
 -   External Tables
 -   Graph Tables
 -   dbo.DimAccount
 -   dbo.DimBranch
 -   dbo.DimCustomer
 -   dbo.FactTransaction
 -   Dropped Ledger Tables

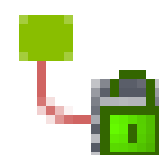
Building the ETL Pipeline with Talend

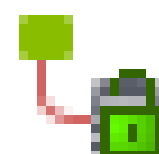
After the DWH foundation was ready, I moved on to Talend Open Studio to build the pipeline that would populate the tables. I created four separate pipelines or 'Jobs' for each task, making the process modular and easier to manage.

Job Designs

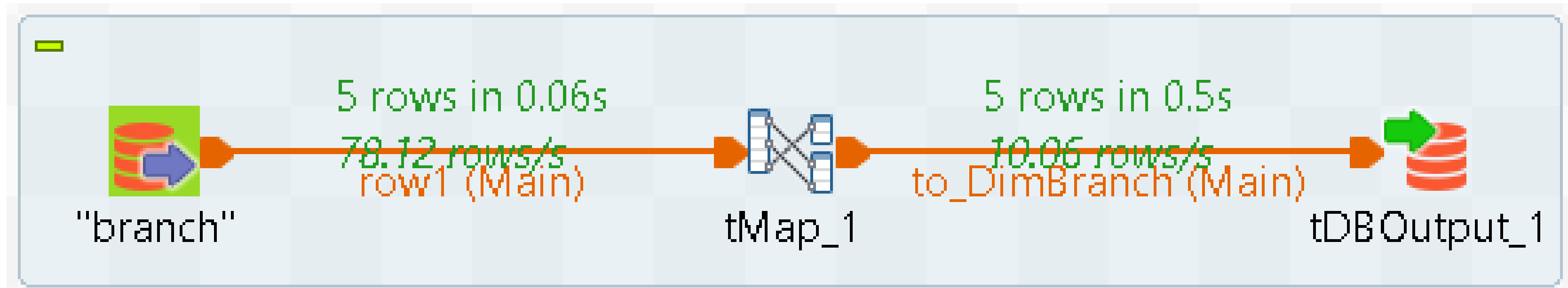
 Load_DimAccount 0.1

 Load_DimBranch 0.1

 Load_DimCustomer 0.1

 Load_FactTransaction 0.1

Loading DimBranch



Starting job Load_DimBranch at 20:26 31/07/2025.

[statistics] connecting to socket on port 3562

[statistics] connected

[statistics] disconnected

Job Load_DimBranch ended at 20:26 31/07/2025. [Exit code = 0]

Job ETL

Loading DimBranch

row1	
Column	
 branch_id	
branch_name	
branch_location	

to_DimBranch	
Expression	Column
row1.branch_id	 BranchID
row1.branch_name	BranchName
row1.branch_location	BranchLocation

Loading DimBranch

1

SELECT * FROM DWH.dbo.DimBranch;

74 %

✔

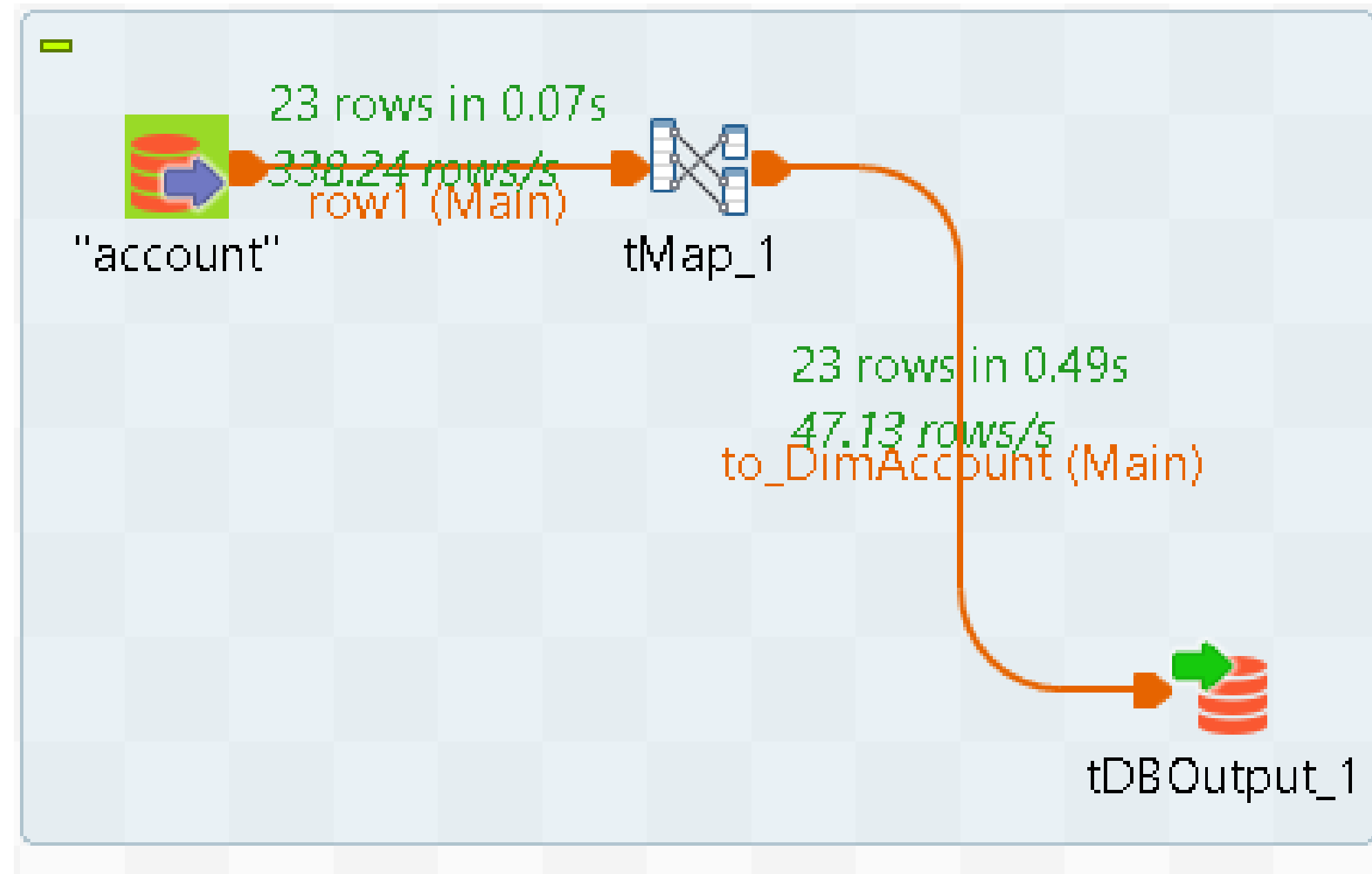
No issues found

Results

Messages

	BranchID	BranchName	BranchLocation
1	1	KC Jakarta	Jl. Gatot Subroto No 13
2	2	KC Bogor	Jl. Padjajaran No 43
3	3	KC Depok	Jl. Raya Sawangan No 34
4	4	KC Tangerang	Jl. Cisauk No 50
5	5	KC Bekasi	Jl. Ahmad Yani No 23

Loading DimAccount



Starting job Load_DimAccount at 20:59 31/07/2025.

[statistics] connecting to socket on port 3712
[statistics] connected
[statistics] disconnected

Job Load_DimAccount ended at 20:59 31/07/2025. [Exit code = 0]

Job ETL

Loading DimAccount

row1	
Column	
account_id	
customer_id	
account_type	
balance	
date_opened	
status	

to_DimAccount	
Expression	Column
row1.account_id	AccountID
row1.customer_id	CustomerID
row1.account_type	AccountType
row1.balance	Balance
row1.date_opened	DateOpened
row1.status	Status

Loading DimAccount

1SELECT * FROM DWH.dbo.DimAccount;

74 %

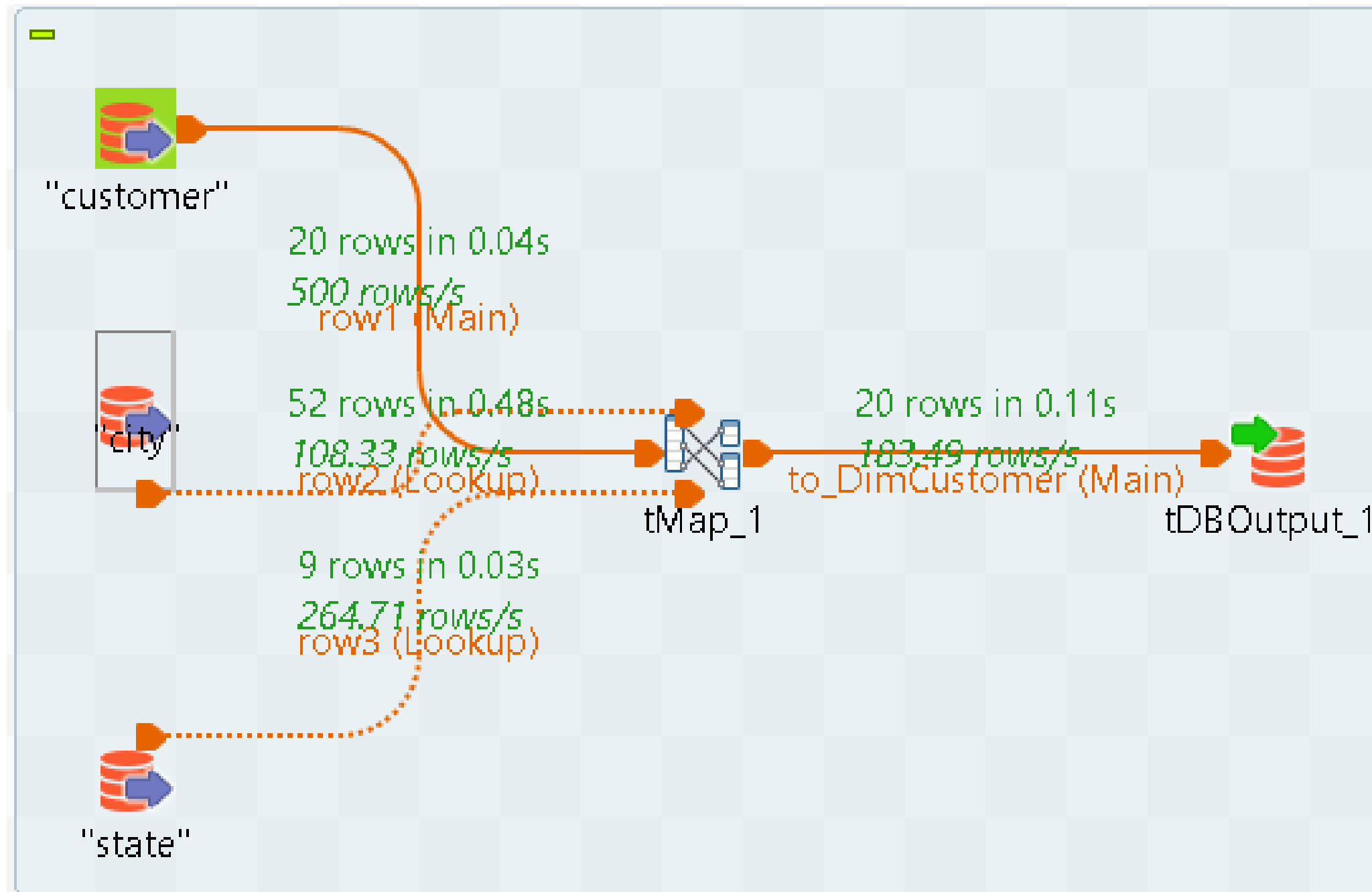
No issues found

Results

Messages

	AccountID	CustomerID	AccountType	Balance	DateOpened	Status
1	1	1	saving	1500000,00	2020-05-01	active
2	2	2	saving	500000,00	2020-06-01	active
3	3	1	checking	25000000,00	2020-06-21	active
4	4	3	checking	4500000,00	2021-06-24	terminated
5	5	4	saving	75000000,00	2020-06-29	active
6	6	5	checking	1500000,00	2020-07-01	active
7	7	6	saving	15000000,00	2020-07-14	terminated
8	8	7	checking	25000000,00	2020-07-15	active
9	9	8	saving	80000000,00	2020-07-15	active
10	10	9	checking	25000000,00	2020-07-16	active
11	11	10	saving	75000000,00	2020-07-24	active
12	12	11	checking	25000000,00	2020-08-08	active
13	13	12	saving	55000000,00	2020-08-15	active
14	14	13	checking	25000000,00	2020-08-15	active
15	15	14	saving	45000000,00	2020-09-25	terminated
16	16	15	checking	25000000,00	2020-09-26	active
17	17	15	saving	10000000,00	2020-10-19	active
18	18	17	checking	25000000,00	2020-10-21	active
19	19	18	saving	55000000,00	2020-11-11	active
20	20	19	checking	25000000,00	2020-11-19	active
21	21	20	checking	6000000,00	2020-11-29	active
22	22	3	checking	4000000,00	2022-08-18	active
23	23	6	checking	5000000,00	2022-01-10	active

Merging Customer Data (Load_DimCustomer)



Starting job Load_DimCustomer at 21:32 31/07/2025.

```
[statistics] connecting to socket on port 3447  
[statistics] connected  
[statistics] disconnected
```

Job Load_DimCustomer ended at 21:32 31/07/2025. [Exit code = 0]

Merging Customer Data (Load_DimCustomer)

↑ ↓ | 📄 🖨

row1 🛠️ ➕ 🔍 🖨

Column
🔑 customer_id
customer_name
address
city_id
age
gender
email

Find : ⬇ ⬆ 🔍

Var 🛠️ ✖ ⬆ ⬇ 🖨

➕ ✖ ⬆ ⬇ | 🖨 | Auto map!

to_DimCustomer 🛠️ ➕ 🔍 🖨

Expression	Column
row1.customer_id	🔑 CustomerID
StringHandling.UPCASE(row1.customer_name)	CustomerName
StringHandling.UPCASE(row1.address)	Address
row1.age	Age
StringHandling.UPCASE(row1.gender)	Gender
row1.email	Email
row2.city_name	CityName
row3.state_name	StateName

Merging Customer Data (Load_DimCustomer)

1

SELECT * FROM DWH.dbo.DimCustomer:

74 %

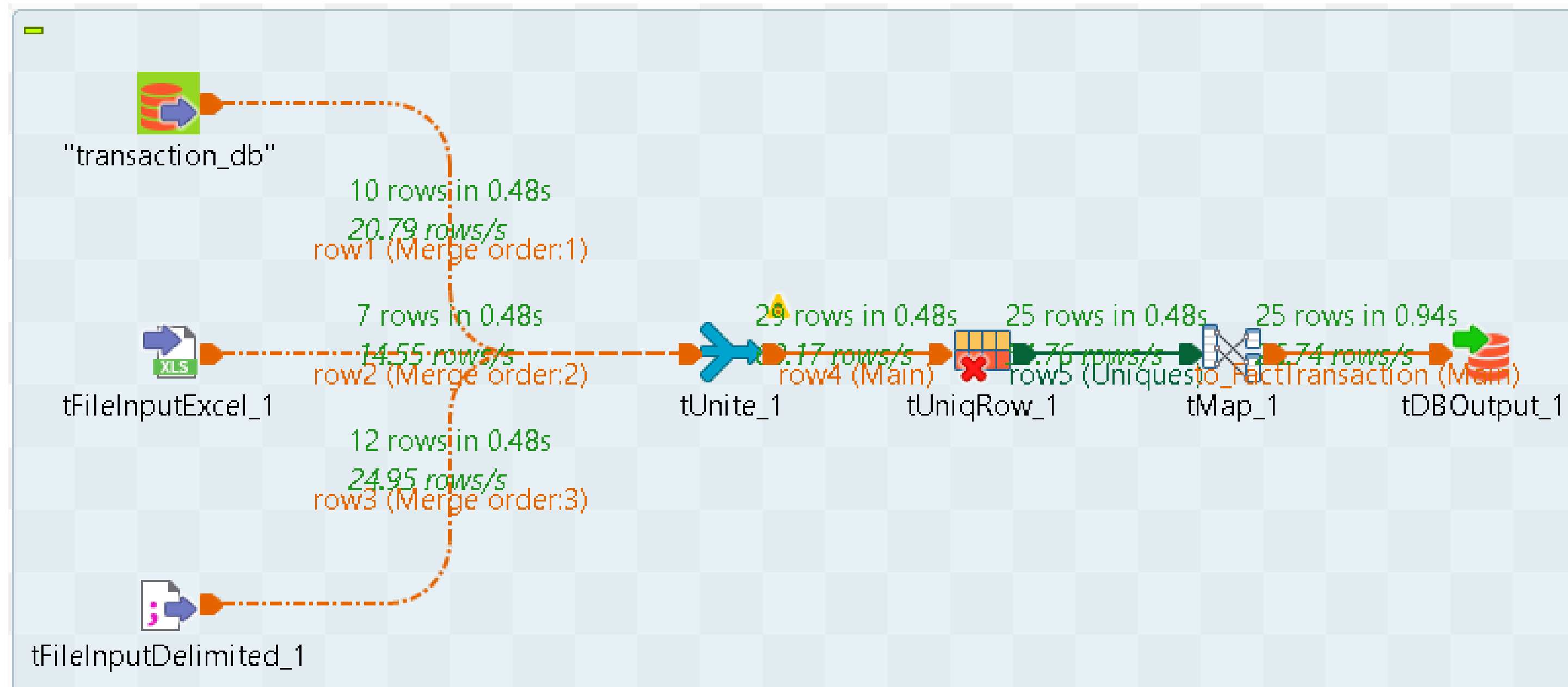
No issues found

Results

Messages

	CustomerID	CustomerName	Address	CityName	StateName	Age	Gender	Email
1	1	SHELLY JUWITA	JL. BOULEVARD NO. 31	Kelapa Gading	Jakarta Utara	25	FEMALE	shelly@gmail.com
2	2	BOBI RINALDO	JL. MANGGA NO. 1	Tanjung Priok	Jakarta Utara	31	MALE	Bobi@gmail.com
3	3	ADAM MALIK	JL. KINCIR ANGIN NO. 50	Pademangan	Jakarta Utara	23	MALE	Adam@gmail.com
4	4	SUSI RAHMAWATI	JL. KENANGA NO. 11	Cilandak	Jakarta Selatan	45	FEMALE	Susi@gmail.com
5	5	DIMAS PRASETYO	JL. NIAGARA NO. 69	Jagakarsa	Jakarta Selatan	32	MALE	Dimas@gmail.com
6	6	AJI PANGESTU	JL. SEMPURNA NO. 2	Mampang Prapatan	Jakarta Selatan	40	MALE	Aji@gmail.com
7	7	BUNGA MALIKA	JL. MERAK NO. 10	Cibinong	Bogor	27	FEMALE	Bunga@gmail.com
8	8	RIA ADDINI	JL. ARJUNA NO. 40	Menteng	Jakarta Pusat	29	FEMALE	Ria@gmail.com
9	9	LISA WULANDARI	JL. AMPERA NO. 39	Tebet	Jakarta Selatan	26	FEMALE	Lisa@gmail.com
10	10	RIO WIJAYA	JL. ABDUL MUIS NO. 70	Gambir	Jakarta Pusat	52	MALE	Rio@gmail.com
11	11	AHMAD RIYANSAH	JL. KECABI NO. 5	Babelan	Bekasi	27	MALE	Ahmad@gmail.com
12	12	MARIO ROBERT	JL. KEBAYORAN NO. 13	Cisauk	Tangerang	28	MALE	Mario@gmail.com
13	13	KHANSA AUDYA	JL. KEDOYA NO. 50	Cinere	Depok	32	FEMALE	Khansa@gmail.com
14	14	CINTYA GABRIELA	JL. PINTU AIR NO. 14	Kalideres	Jakarta Barat	24	FEMALE	Cintya@gmail.com
15	15	ANI NURAINI	JL. CIMPEDAK NO. 50	Cibitung	Bekasi	45	FEMALE	Ani@gmail.com
16	16	AGUNG MULYONO	JL. DAAN MOGOT NO. 60	Palmerah	Jakarta Barat	38	MALE	Agung@gmail.com
17	17	RIAN WIBOWO	JL. BATU NO. 49	Bojonggede	Bogor	41	MALE	Rian@gmail.com
18	18	LUTFI AULIA	JL. ELANG NO. 2	Ciawi	Bogor	30	MALE	Lutfi@gmail.com
19	19	MALIKA CANTIKA	JL. IMAM BONJOL NO. ...	Balaraja	Tangerang	38	FEMALE	Malika@gmail.com
20	20	RATNA SARIASIH	JL. JAMBORE NO. 41	Sukatani	Bekasi	53	FEMALE	Ratna@gmail.com

Integration Pipeline: Unifying Transaction Data



```
Starting job Load_FactTransaction at 23:48 31/07/2025.  
[statistics] connecting to socket on port 3339  
[statistics] connected  
[statistics] disconnected  
  
Job Load_FactTransaction ended at 23:48 31/07/2025. [Exit code = 0]
```

Job ETL

Integration Pipeline: Unifying Transaction Data

Schema from tMap_1 output

Column	K...	Type	☒ N..	Date Pattern (Ctr...	Length	Precision	Default	Comment
 TransactionID	☒	int	☐		10	0		
AccountID	☐	Integer	☒		10	0		
TransactionDate	☐	Date	☒	"dd-MM-yyyy"	19	0		
Amount	☐	Integer	☒		10	0		
TransactionType	☐	String	☒		50	0		
BranchID	☐	Integer	☒		10	0		

Integration Pipeline: Unifying Transaction Data

Schema

Built-In

Edit schema

Sync columns

Unique key

Column	☐ Key attribute	☐ Case sensitive
transaction_id	☒	☐
account_id	☐	☐
transaction_date	☐	☐
amount	☐	☐
transaction_type	☐	☐
branch_id	☐	☐

Integration Pipeline: Unifying Transaction Data

1

2

SELECT COUNT(*) FROM DWH.dbo.FactTransaction;

SELECT TOP 10 * FROM DWH.dbo.FactTransaction;

74 %

✓

No issues found

Results

Messages

(No column name)

125

	TransactionID	AccountID	TransactionDate	Amount	TransactionType	BranchID
1	1	1	2024-01-17 09:10:00.000	100000,00	Deposit	1
2	2	2	2024-01-17 10:10:00.000	1000000,00	Deposit	1
3	3	3	2024-01-18 08:30:00.000	10000000,00	Transfer	1
4	4	3	2024-01-18 10:45:00.000	1000000,00	Withdrawal	1
5	5	5	2024-01-18 11:10:00.000	200000,00	Deposit	1
6	6	6	2024-01-18 13:10:00.000	50000,00	Withdrawal	1
7	7	6	2024-01-19 14:00:00.000	100000,00	Payment	1
8	8	7	2024-01-19 09:10:00.000	5000000,00	Deposit	1
9	9	8	2024-01-19 10:40:00.000	300000,00	Withdrawal	2
10	10	9	2024-01-20 12:10:00.000	2000000,00	Deposit	1

Simplifying Data Access with Stored Procedures

Once the Data Warehouse was populated, the ETL process was complete. The final step was to create a "shortcut" for analysts. I created two Stored Procedures in SSMS to provide quick access to the most frequently needed data summaries.

Stored Procedure #1 - sp_DailyTransaction

```
1  USE DWH;
2  GO
3
4  CREATE PROCEDURE sp_DailyTransaction
5      @start_date DATE,
6      @end_date DATE
7  AS
8  BEGIN
9
10     SET NOCOUNT ON;
11
12     SELECT
13         CAST(TransactionDate AS DATE) AS [Date],
14         COUNT(TransactionID) AS TotalTransactions,
15         SUM(Amount) AS TotalAmount
16     FROM
17         FactTransaction
18     WHERE
19         CAST(TransactionDate AS DATE) BETWEEN @start_date AND @end_date
20     GROUP BY
21         CAST(TransactionDate AS DATE)
22     ORDER BY
23         [Date];
24 END;
25 GO
```

Stored Procedure #1 - sp_DailyTransaction

1

EXEC sp_DailyTransaction @start_date = '2024-01-18', @end_date = '2024-01-20';

67 %

✖

1

⚠

0

↑

↓

◀

Results

Messages

	Date	TotalTransactions	TotalAmount
1	2024-01-18	4	11250000,00
2	2024-01-19	3	5400000,00
3	2024-01-20	4	4000000,00

Stored Procedure #2 - sp_BalancePerCustomer

```
1 CREATE PROCEDURE sp_BalancePerCustomer
2     @customer_name VARCHAR(100)
3 AS
4 BEGIN
5     SET NOCOUNT ON;
6     WITH TransactionSummary AS (
7         SELECT
8             AccountID,
9             SUM(
10                 CASE
11                     WHEN TransactionType = 'Deposit' THEN Amount
12                     ELSE -Amount
13                 END
14             ) AS TotalTransactionAmount
15         FROM
16             FactTransaction
17         GROUP BY
18             AccountID
19     )
20     SELECT
21         c.CustomerName,
22         a.AccountType,
23         a.Balance AS InitialBalance,
24         a.Balance + ISNULL(ts.TotalTransactionAmount, 0) AS CurrentBalance
25     FROM
26         DimCustomer c
27     JOIN
28         DimAccount a ON c.CustomerID = a.CustomerID
29     LEFT JOIN
30         TransactionSummary ts ON a.AccountID = ts.AccountID
31     WHERE
32         c.CustomerName LIKE '%' + @customer_name + '%'
33         AND a.Status = 'active';
34 END;
35 GO
```

Stored Procedure #2 - sp_BalancePerCustomer

1

`EXEC sp_BalancePerCustomer @customer_name = 'Shelly';`

67 %

 1

 0







 Results

 Messages

	CustomerName	AccountType	InitialBalance	CurrentBalance
1	SHELLY JUWITA	saving	1500000,00	1600000,00
2	SHELLY JUWITA	checking	25000000,00	14000000,00

Thank You

